

Day 49: Weekly Revision

Day 49 — Weekly Revision

1. Day Number

Day 49

2. Topic Name

Revision of Days 43–48

Today you will revise the OOP concepts you learned during the last six days:

class → **object** → **constructor** → **attributes** → **methods** → **basic encapsulation**

3. Connection

During Days 43–48, you gradually moved from storing simple data to building objects that contain both **data and behavior**.

The journey looked like this:

```
User data
  ↓
Class
  ↓
Object
  ↓
Constructor
  ↓
Attributes
  ↓
Methods
  ↓
Deposit / Withdraw
  ↓
Basic Encapsulation using _balance
```

Today you will combine these ideas in one very small BankAccount exercise.

4. Revision Summary of Days 43–48

Day 43 — Class and Object

You learned that a **class is a blueprint**.

For example:

```
class User:
    ...
```

An **object** is an actual instance created from that blueprint.

Conceptually:

```
User class
  ↓
User object
  ↓
name = "Amit"
password = "abc123"
```

Day 44 — Constructor `__init__()`

You learned to initialize object data when an object is created.

Example structure:

```
def __init__(self, name):
    self.name = name
```

The constructor runs automatically when an object is created.

```
Create object
  ↓
__init__() runs
  ↓
Attributes receive values
```

Day 45 — Methods

You learned that a class can contain functions called **methods**.

A method describes something an object can do.

For example:

```
BankAccount

Data:
account_holder
balance

Behavior:
display_account()
```

Day 46 — Deposit Method

You added behavior that changes account balance.

The important idea was:

```
Receive deposit amount
  ↓
Is amount positive?
  /      \
```

Yes ↓ Update balance	No ↓ Don't update
-------------------------------	-------------------------

Day 47 — Withdraw Method

You learned that a method can contain validation before changing an attribute.

For withdrawal, you checked things such as:

amount > 0
AND
amount <= balance

Only valid withdrawals should change the balance.

Day 48 — Basic Encapsulation

You changed:

balance

to:

_balance

The leading underscore means:

This attribute is intended for internal use inside the class.

You also learned the idea of accessing the balance through a method such as:

get_balance()

instead of encouraging direct changes everywhere.

5. Important Topics

The most important concepts to remember are:

Concept	Meaning
class	Blueprint for creating objects
object	Actual instance of a class
__init__()	Constructor used to initialize an object
self	Refers to the current object

Concept	Meaning
attribute	Data stored inside an object
method	Function belonging to a class
<code>_balance</code>	Attribute intended for internal use

The relationship is:

```

Class
├── Attributes → store data
└── Methods → work with that data

```

6. Foundational Notes

Class

A class groups related **data and behavior** together.

Imagine a bank account.

It has data:

```

Account holder
Balance

```

And it can perform actions:

```

Deposit
Withdraw
Display balance

```

OOP allows us to keep these things together.

Object

Suppose BankAccount is the blueprint.

You could create:

```

Account 1
Holder: Amit
Balance: 5000

Account 2
Holder: Neha
Balance: 8000

```

Both objects come from the same class but store their own data.

self

Inside a class, `self` means:

| the current object.

For example:

```
self.name
```

means:

```
the name belonging to this particular object
```

If two different objects exist, each can have a different `self.name`.

Attributes

Attributes store object data.

For example:

```
self.name  
self._balance
```

Here:

```
name      → account holder  
_balance  → current account balance
```

Methods

Methods work with object data.

For example, a deposit method conceptually does:

```
Read amount  
  ↓  
Validate amount  
  ↓  
Change _balance
```

Why `_balance`?

Writing:

```
_balance
```

does **not** make the variable completely private in Python.

Instead, the underscore is a convention meaning:

| This value should normally be handled through the class's methods rather than changed directly from outside.

So prefer thinking like:

```
Outside code
  ↓
deposit()
  ↓
validation
  ↓
_balance updated
```

rather than:

```
Outside code
  ↓
directly changes _balance
```

This is the beginning of **encapsulation**.

7. Easy Example

Consider a simple wallet class.

It could have:

```
owner
_money
```

and one method:

```
add_money()
```

Suppose:

```
Owner = Ravi
Money = 100
```

Then Ravi adds 50.

The method checks:

```
Is 50 positive?
  ↓
Yes
  ↓
_money = 100 + 50
  ↓
150
```

The important idea is not the wallet itself.

The important idea is:

Object stores data
+
Method safely changes data

8. Revision Problem Statement

Create a class named:

BankAccount

The class should store:

account holder
balance

Store the balance using:

_balance

Add **only one method**:

deposit()

The deposit() method should add money to _balance only when the deposit amount is positive.

Then:

Create one BankAccount object
↓
Deposit one amount
↓
Check the updated account balance

Do not add withdraw() today because this exercise is intentionally small and focused on revision.

9. Concepts Used

This problem combines the core OOP ideas from the week:

```
class
↓
__init__()
↓
self
↓
attributes
↓
object
↓
```

```
method
  ↓
condition
  ↓
attribute update
  ↓
basic encapsulation
```

You will especially practice the relationship between:

```
self._balance
```

and:

```
deposit()
```

10. Thought Process

Before writing Python, think about the problem in small pieces.

First ask:

What object am I representing?

```
Bank account
```

Then ask:

What data should each bank account store?

```
account holder
balance
```

Then ask:

What should happen when the object is created?

The constructor should receive the starting information and store it in attributes.

Then ask:

What action can this account perform?

For today's problem:

```
deposit money
```

Then ask:

Should every deposit be accepted?

No.

Only:

```
amount > 0
```


should update the balance.

Finally:

```
old balance + valid deposit
      ↓
    new balance
```

11. Pseudocode

```
START

CREATE BankAccount class

    CREATE constructor with account holder and balance

        STORE account holder

        STORE balance in _balance

    CREATE deposit method with amount

        IF amount is greater than 0

            ADD amount to _balance

        ELSE

            DO NOT update _balance

CREATE one BankAccount object

CALL deposit method with one amount

CHECK the updated balance

END
```

Notice the important flow:

```
Object
  ↓
deposit(amount)
  ↓
amount > 0 ?
 /      \
Yes      No
  ↓      ↓
Update  Keep
_balance balance unchanged
```

12. Suggested Solving Approach

Use an **OOP-based approach**.

Start with the class definition.

Then create the constructor.

Inside the constructor, create your two attributes:

```
account holder
_balance
```

Next, create the `deposit()` method.

Inside that method, perform validation before changing `_balance`.

After the class is complete, create one object and call its `deposit()` method.

Think of the structure as:

```
CLASS DEFINITION
|
├── __init__()
│   ├── account holder
│   └── _balance
└── deposit()
    ├── validate amount
    └── update _balance

    ↓

CREATE OBJECT

    ↓

CALL deposit()
```

13. Easy Edge Cases

The two main edge cases are **deposit 0** and **negative deposit**.

For deposit 0:

```
Current balance = 1000
Deposit = 0

0 > 0 ?
No

Balance remains 1000
```

For a negative deposit:

```
Current balance = 1000
Deposit = -500

-500 > 0 ?
No

Balance remains 1000
```

For comparison, a valid deposit behaves like:

```
Current balance = 1000
Deposit = 500

500 > 0 ?
Yes

New balance = 1500
```

14. Common Mistakes to Avoid

Be careful not to forget `self` in the constructor or method definitions, confuse a local variable such as `balance` with the object's `self._balance`, update the balance before validating the deposit amount, use `>= 0` when the requirement says the amount must be **positive**, or accidentally create a local variable such as `_balance` instead of updating `self._balance`.

Also remember that calling a method requires an object:

```
account object
    ↓
deposit method
```

and each object's `_balance` belongs to that particular object.

15. Quick Self-Check Questions

1. What is the difference between a **class** and an **object**?
2. Why do we use `self` inside a class?
3. When does `__init__()` execute?
4. Why are we using `_balance` instead of simply `balance`?
5. If `_balance` is 1000 and `deposit(300)` is valid, what should the new balance become?

If these answers feel clear, you understand the main ideas from Days 43–48.

16. Hint Only

Build the problem in this order:

```
BankAccount class
    ↓
```

```
__init__(holder, balance)
↓
store holder
↓
store balance as self._balance
↓
deposit(amount)
↓
check amount > 0
↓
update self._balance
↓
create object
↓
call deposit once
```

Small syntax hint

Inside your deposit method, think about a condition shaped like:

```
if _____ > 0:
    self._balance = _____
```

Your job is to determine what belongs in the blanks.

Do not add `withdraw()`, `get_balance()`, or other methods for this revision exercise. The goal is to prove that you understand the basic OOP flow with one simple behavior.